# Expected Linear Time Algorithm for Computing 3D Relative Neighborhood Graphs

Z. Zzyzek[a,*,1] (Researcher)

ARTICLE INFO

*Keywords*:
graph algorithm
relative neighborhood graph
computation geometry
RNG

ABSTRACT

An expected linear time algorithm for computing the relative neighborhood graph (RNG) for points in Euclidean space, distributed uniformly in a unit three dimensional cube, is given. For each point, potential neighbors for an RNG connection are limited to an expected small bounding volume. Volumes relative to the considered point are bounded through half plane partitions, constructed from the point and its neighbors, that remove regions where no RNG edge connections to the point can occur. The small bounding volume provides an expected constant number of neighbors to consider, making each single point RNG calculation $O(1)$, providing the $O(n)$ expected runtime when enumerating through all points. To the author's knowledge, this improves on the previously best known $O(n \log n)$ in three dimensions.

## 1. Introduction

Let $P = (p_0, p_1, \ldots, p_{n-1})$ be a set of points in $\mathbb{R}^3$. The relative neighborhood graph, RNG($P$), is an undirected, simple graph that connects two vertices, $p_i, p_j$ if and only if there is no other vertex within the *lune* between them.

$$(p_i, p_j) \in \mathrm{RNG}(P) \iff |p_i - p_j| \leq \max_{k \neq i,j}(|p_i - p_k|, |p_j - p_k|)$$

Here, $|\cdot|$ is the standard Euclidean norm. This paper focuses almost exclusively on points in 3D but will occasionally talk about points in 2D for explanatory examples.

Figure ?? shows a highlighted 2D example of a connections and precluded connection and Figure ??, ?? shows examples of a relative neighborhood graph for 2D and 3D.

When points are placed uniformly at random in the unit cube, the maximum degree of the relative neighborhood graph is bounded by a constant. When random in this manner, the edges from any point connecting it to its relative neighborhood graph only extend to a proportionally small local neighborhood. This local neighborhood is effectively constant in neighbor size, allowing us to calculate each points relative neighborhood graph edges in constant time, providing a linear time algorithm to find the relative neighborhood graph in 3D.

It's believed that the worst case runtime for a relative neighborhood graph in 2D and 3D is at least $O(n \log n)$. Randomness holds special structure and many cases in the general case won't happen, with high probability. Note that randomness is a different condition than generic or general position, as point configurations that pass a generic or general position criteria might not be considered random. Only points uniformly distributed in the unit cube are considered in this paper, without focusing on whether they're in generic or general position.

zzyzek@thumpernet.com (Z. Zzyzek)
zzyzek.github.io (Z. Zzyzek)
ORCID(S): 0009-0005-2175-1166 (Z. Zzyzek)

## 2. Related Work

OUTLINE

- Talk about KN85 and KNT86 and their exceptions to corner and side regions of grid. Also talk about restriction to 2d and how generalizations to 3d are complicated

- 3d algs of agarwal,matousek are meant for worst case setups but super linear (e.g. advesarial)

- talk about efficient computation of Urquhart (sp?) graph

- talk about using delaunay triangulation to bootstrap

- review gives best known algorithm at $O(n \log n)$ for general position (doesn't talk about random position?)

- Jaromczyk and Kowaluk on $O(n^2 \log n)$ 3d algorithm with 'coarse' elimination phase that was inspiration for plane cut heuristic

## 3. Algorithm

All points eventually will be evaluated through a naive relative neighborhood graph calculation, relative to an *anchor point*, $p$. Algorithm 1 provides the pseudo code that does the all pairs test of points in $Q$ to the anchor point, $p$.

Algorithm 1 has $O(|Q|^2)$ runtime. As will be detailed in a later section, $|Q|$ will be shown to be almost surely bounded above by a constant, making the naive pairwise point testing of Algorithm 1 constant time.

Algorithm 2 intializes the grid and then processes each point individually with a call to the Algorithm 3.

**Algorithm 1** Single point naive Relative Neighborhood Graph for the list $Q$ to test for connections to $p$

```
1: procedure RNGPointNaive(Q, p)
2:     for q ∈ Q/{p} do
3:         isConnected = False
4:         for u ∈ Q/{p, q} do
5:             if InLune(p, q, u) then
6:                 isConnected = False
7:         if isConnected == True then E = E ∪ (p, q)
8:     return E
```

**Algorithm 2** Relative Neighborhood Graph, Secure Posts on Increasing Fence (SPoIF)

```
1: procedure RNGSPoIF(P)
2:     E = {}
3:     N = |P|,   N_s = ⌈N^(1/3)⌉,   s = 1/N_s,
4:     B[0 : N_s][0 : N_s][0 : N_s] = ∅
5:     for p ∈ P do
6:         append(B[⌊N_s p_x⌋][⌊N_s p_y⌋][⌊N_s p_z⌋], p)
7:     for p ∈ P do
8:         E = E ∪ RNGPointSPoIF(B, P, p)
9:     return E
```

**Algorithm 3** Single point Relative Neighborhood Graph, Secure Posts on Increasing Fence

```
1: procedure RNGPointSPoIF(B, P, p)
2:     Initialize FencePostCluster[] entries to False
3:     Q = {},   R_f = 0,   R_max = 0
4:     c = grid cell p falls in
5:     for (R_f ≤ N_s) and
6:         (False ∈ FencePostCluster) do
7:         Q' = GridPerimeterCellPoints(B, p, R_f)/{p}
8:         for q ∈ Q' do
9:             Q = Q ∪ {q}
10:            R' = max grid cell distance from c
11:                 Lune(p, q) falls in
12:            R_max = max(R_max, R')
13:        for q ∈ Q' do
14:            Define ρ(u) = sgn((p − q) · (u − q))
15:            Define σ(u) = sgn(u · (u − q))
16:            for every cluster, γ do
17:                if γ is out of bounds then
18:                    FencePostCluster[γ] = True
19:                else if every v ∈ γ,
20:                        ρ(v) = −ρ(p) and
21:                        σ(v) = −σ(p) then
22:                    FencePostCluster[γ] = True
           R_f = R_f + 1
23:    for R_e = [R_f + 1 : R_max] do
24:        Q'' = GridPerimeterCellPoints(B, p, R_e)
25:        for q ∈ Q'' do
26:            Q = Q ∪ {q}
27:    return RNGPointNaive(Q, p)
```

Algorithm 3 processes each point individually by looking at a local grid neighborhood of the grid, and testing nearby points to see if they've secured the fence. The fence securitization is done by testing that all points in the frusturm that make the fence face patch, called *fence posts*, are *secured*. The vector in the direction of $(p-q)$ and the fence post point relative to the grid cell that $p$ lies in are tested against each other to make sure the fence post points lie on the opposite side of the implied half plane cut defined by the normal of $(p-q)$ and will do so as the fence grid grows.

Once the fence has been secured, extra points past the fence need to be added to ensure that all nearby points that might affect connections to the anchor point are included to the naive relative neighborhood graph calculation. The $R_{\max}$ is calculated empirically during the course of Algorithm 3 but is bounded above by a constant factor (see Appendix **??**).

After fence securitization and all nearby relevant points have been collected, a call to the naive relative neighborhood graph, on the collected points relative to the anchor point, is called in Algorithm 1.

The auxiliary function, *GridPerimeterCellPoints*, returns points within the grid, $B$, centered at the grid cell that $p$ resides in and that lie an integral $R$ distance away. The list of points within the enumeration the shell of cells at distance $R$ is returned, with cells falling completely out of bounds skipped over. A small note that the initial call of *GridPerimeterCellPoints* needs to exclude anchor point, $p$, as the starting grid cell will include the anchor point in question.

In the worst case, Algorithm 3 will iterate until it encompases all points within the grid when the *FencePostCluster* has fallen completely out of bounds. In such a case, Algorithm 1 is run on all points and is equivalent to the degenerate condition of running a naive $O(N^2)$ relative neighborhood graph computation on all neighbor points to an anchor point.

For random point distribution in the unit cube, we expect much better runtime as the the expected number of potential neighbors collected will be bounded.

## 4. Motivation and Structure

This section provides proofs that justify the algorithm's correctness and runtime. Before going through specifics, an overview is provided, giving motivation for the technical details that follow.

From the problem formulation, there are $N$ 3D points chosen uniformly at random in the unit cube. As an initial pass, the $N$ points are then binned into grid cells of size $N^{-1/3}$.

A single point is considered at a time and the relative neighborhood graph connections are calcualted relative to this point. Each point considered in this way is called an *anchor point*. The runtime construction for the relative neighborhood graph relative to each anchor point will be constant, providing an expected $O(N)$ algorithm once each anchor point is processed.

To construct the relative neighborhood graph relative to the anchor point, the algorithm makes use of a *plane partition heuristic*.

For each anchor point, $p$, a nearby point, $q$, is used to create a half plane cut. The half plane cut is defined from the plane that passes through $q$ with normal in the direction of $(p-q)$, $H_{q,p} = \{u : (p-q)\cdot(u-q) > 0\}$. All points that fall outside of $H_{q,p}$ can never have an edge to $p$ as the point $q$ would be in the lune excluding the connection.

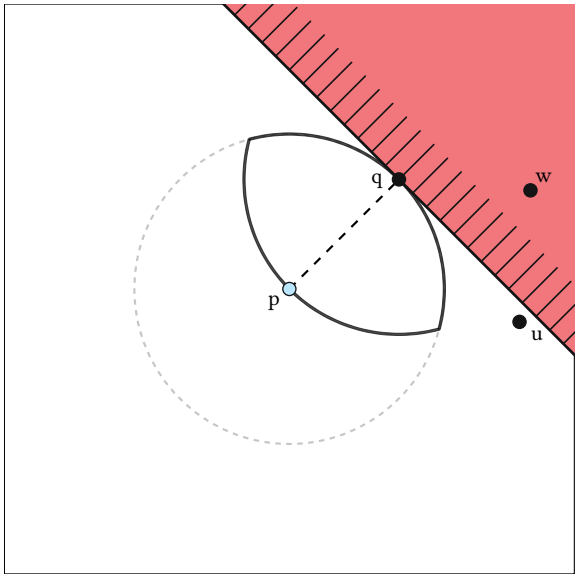


**Figure 1**: 2D representation of the plane partition heuristic. $p$ is the anchor point with $q$ a nearby point used to create the plane partition. The point $w$ is excluded by the plane partition heuristic as $q$ would be in the lune of $p$ and $w$. The point $u$ is not excluded by the plane partition heuristic, even if $q$ might fall within the lune created by $p$ and $u$ because $u$ falls below the plane partition.

Figure 4 provides a 2D representation of the plane partition heuristic, showing the anchor point $p$, a nearby point $q$, a point that is excluded by the heuristic, $u$, and a point, $w$, that still has a potential relative neighborhood graph connection to $p$.

If, for set of $q$ points, a compact convex hull can be created from the $H_{q,p}$ half plane cuts, then only points within the convex hull need be considered for relative neighborhood graph connections to $p$.

Instead of working with the convex hull directly, a bounding region is used, called a *fence*, for which constructing a bounding convex hull is simpler. The fence is a cube, centered at the grid cell center that the anchor point $p$ occupies. The fence initially has side length of the cube cell spacing. Each fence face is broken into square regions, called *fence patches*, and each patch is tested against the half plane cut. In this paper, only nine fence posts per face are considered, with fence post patches clustered into $2\times 2$ posts, four clusters per fence face.

If the fence patch is completely outside of the half plane cut, the fence patch is said to be *secured*. Figure **??** shows various patches that are in different states of securitization from half plane cuts.

Points within the fence are used to construct the half plane cuts. If the half plane cuts fail to secure the fence, the fence cube side is increased, staying on grid cell boundaries. The process is then repeated, attempting to secure the fence from the newly added points. The fence is increased until it is secured, in the worst case increasing to the degenerate condition that subsumes the whole unit cube.

Once all fence patches are secured, there is an implicit compact convex hull that is completely within the fence cube and only points that fall within the fence need be considered for relative neighborhood graph connections to the anchor point $p$. Though only points inside of the fence cube have the possibility of a relative neighborhood graph connection to $p$, points outside of the fence can still have an effect by potentially restricting other connections to $p$.

The fence needs to be extended to add these additional points. As will be shown below, the fence need only be extended by a constant proportion to the fence side length. This adds an expected constant factor of points to consider, leaving the linear runtime unaffected.

Once a fence has been secured and all relevant nearby points collected, a naive relative neighborhood graph algorithm is then used to find connections from the anchor point to its nerby point collection. The size of collected nearby points is expected linear, so the naive algorithm runs in constant time.

Figure **??** shows a graphical cartoon of this process as a summary.

The fence patch is reprsented by four points on its corners, called *fence posts*. A fence post is secured if the patch, or *cluster* of fence posts, falls completely on the restricted side of a half plane cut.

As the algorithm operates by securing posts on an ever increasing fence until all posts secured, the algorithm is named Securing Posts on Increasing Fence (SPoIF).

Proofs of correctness and algorithmic details are presented in section TK, TK. Before proceeding with the technical aspects of the algorithm, a few points are discussed.

A secured fence provides a compact convex hull, the fence cube, and thus implies a smaller compact convex hull bounded within it. The converse is not true and a

convex hull constructed from an anchor point and its neighbors need not be secured by a fence in this way. The convex hull could be considered directly, instead of the fence structure as proxy, but this involves using convex hull planes defined by a point and normal with the need to derive the vertex intersection points. Since there are an expected constant number of half planes to consider, vertex enumeration conversion would be constant but at the cost of an inflated runtime factor with a naive algorithm. Using the convex hull directly is not considered further in this paper.

Below, nine fence posts per cube face (thirty six in total), will be used. Some other number of fence posts could be used. Another alternative is to use a shrinking area on each of the faces, defined by the half plane intersections.

These are mentioned to note that many other methods could work and still retain the expected linear time runtime. Other than to mention them as possibilities, these ideas are not pursued further in this paper.

The nine fence posts per fence face implementation is now considered in detail.

### 4.1. Overview

In this section, we provide some technical proofs providing correctness for the algorithm in section TK. The general strategy will be to show that only nearby points within a compact convex hull of an anchor point need be considered for relative neighborhood graph connections, that nearby points within a convex factor distance of the convex hull need to be considered for any influence and that the approximation by the coarse fence and fence posts imply a contained compact convex hull.

Let $P = (p_0, p_1, \ldots, p_{n-1})$, where each $p_i \in \mathbb{R}^3$ is chosen uniformly at random in the unit cube.

A rough overview of the algorithm proceeds as follows:

- **Initialize** a three dimensional grid, where each cell size of $n^{-1/3}$ on a side
- **Bin** points into grid cells, allowing for multiple points in a bin.
- For each point, $p$:
  - **Add** nearby points from adjacent grid cells
  - **Increase** the radius of a bounding cube, at grid cell increments, until the collected nearby points have precluded connections from all other points outside bounding cube to $p$
  - **Extend** the list of nearby points, once a bounding cube has been found, to include points that might preclude connections on the inner bounding cube
  - **Run** a naive relative neighborhood graph, $\mathrm{RNG}(P, p)$, on all collected nearby points to determine connections to $p$

The binning phase creates a Poisson process with an average of one point per bin. A coarse grained plane partition heuristic is used to exclude further away points from consideration and is the method by which the process knows to stop extending the bounding cube.

Once a bounding cube has been found, other points outside of the bounding cube, but still within a, larger, local distance need to be added as they could influence connections on the interior bounding cube. The list of all neighbors is given to a naive relative neighborhood graph algorithm and the process is then repeated for each point.

Given points $p$ and $q$, any point, $w$, that lies on the other side of the plane with normal $\frac{(q-p)}{|q-p|}$ that passes through $q$, cannot connect to $p$, as $q$ would be in any lune that $p$ and $w$ would create. Figure **??** gives a graphical representation of points excluded by this plane heuristic in two dimensions.

The entire region that $q$ excludes from $p$ is more complicated (see Figure **??**) so the plane partition heuristic is coarse but provides an easy way to exclude large portions of points.

To maintain a linear time algorithm, large collections of points that can never connect to our anchor point, $p$, need to be efficiently excluded. To do this, consider a current grid cell collection of integral radius $R \in \mathbb{Z}$, centered on the grid cell where anchor point $p$ resides. If the tangent planes that passes through neighbor $q$, and in direction $(q-p)$ within the grid cell collection, partition each of the six sides of our cube of radius $R$, we can be assured that any points outside of the grid cell collection will never be able to connect to $p$.

A grid cell collection of a radius $R$ is called a *fence* of radius $R \in \mathbb{Z}$. When $R$ is understood from context, this will be called just a *fence*. If the sides of the *fence* has been partitioned, with anchor point $p$ and neighbors $q$ within the *fence*, the *fence* will be called *secured*.

Points outside of the *fence* will never be able to connect to anchor point $p$ but they still might be in the lune of some points within the *fence*. For this reason, when collecting points for the naive relative neighborhood graph calculation, some points outside of the fence will need to be added, as they can sabotage potential edges within the *fence*.

This will be done by considering a larger radius of the current *fence* to add points but the increase in $R$ will shown to be bounded by a constant factor, allowing overall linear time to be maintained.

Before discussing the algorithm in detail, preliminary structural results will be discussed.

### 4.2. Foundation

In this section we go through the proofs that provide the foundation of correctness for the algorithm.

For an anchor point $p$, if $w$ is on the other side of the plane that passes through $q$, with tangent in the direction of $(q-p)$, then $q$ will always be in the lune created by $p$ and $w$ (see Figure 4):

**Lemma 1 (Plane Partition Heuristic).** *if $p, q, w \in \mathbb{R}^3$ with:*

$$w \in H_{q,p} = \{u \ : \ (p-q)\cdot(u-q) > 0\}$$

*then*

$$q \in Lune(p,w) = \{v \ : \ |v-p| \leq |p-w| \ \& \ |v-w| \leq |p-w|\}$$

Proof. TK

The plane partition heuristic takes points near $p$ to be used in constructing a convex hull around $p$. Only nearby points that fall within the convex hull need to be tested for relative neighborhood graph connections to $p$.

Lemma 2 states this more formally:

**Lemma 2 (Convex Hull Heuristic).** *Let $p \in \mathbb{R}^3$, $Q = (q_0, q_1, \ldots, q_{m-1}) \subset P$ and $\mathbf{C}$ be the compact convex hull created from $H_i = \{u \in \mathbb{R}^3 : (p-q_i)\cdot(u-q_i) > 0\}$, then $(w,p) \in RNG(P,p) \rightarrow w \in \mathbf{C}$*

Proof. By Lemma 1, all $v$ outside of $\mathbf{C}$ will have some $q_i \in Q$ in the Lune$(v,p)$, so cannot have an RNG edge to $p$.

Lemma 2 only guarantees that RNG connections to $p$ fall within the hull, $\mathbf{C}$. Not every point within $\mathbf{C}$ might connect to $p$ and points outside of $\mathbf{C}$ might still influence connections to $p$. For example, Figure **??** shows a potential connection between two points within the convex hull but is thwarted by a point outside the convex hull that is within the lune created by the pair.

Points outside of the convex hall might influence connections to $p$ but we can bound how far these points can be. Nearby points influencing $p$ can only be a constant factor of the maximum distance of $p$ to the convex hull:

**Lemma 3 (Convex Hull Extension Heuristic).** *Let $p$ be the anchor point and $\mathbf{C}$ be the convex hull as detailed above. Let $R_{max} = \max_{u\in\mathbf{C}}|u-p|$ be the maximum distance of the convex hull boundary to $p$. All points that could affect $RNG(P,p)$ lie within a sphere of radius $R_{max}$, centered at $p$.*

Proof. The maximum lune radius for a point, $u$, within the convex hull to $p$ is bounded above by $R_{\max}$. For a point, $w$, to influence a potential connection of $u$ to $p$, $w$ must lie within the lune created by $u$ and $p$. If $|w-p| > R_{\max}$, $w$ falls outside of any lune that $p$ might create with points within the convex hull.

A bounding box cube is used as a proxy for the convex hull. The bounding box cube, called a *fence*, has each face partitioned into regular squares, called *fence face patches* (or just a *fence patch*). Plane partitions are used to partition the fence patch from the anchor point. Once all fence face patch portions have been partitioned from $p$, a process that will be called *securing the fence*, this implies a compact convex hull contained within the fence.

A secured fence necessarily implies a compact convex hull embedded within the fence but the other direction is not true in general, as there can be a convex hull whose volume is completely contained within the fence but whose partition planes do not secure the fence. We will see that this has no effect on the expected linear runtime, as securing the fence for an individual anchor point is expected to use a constant number of neighbors.

**Theorem 4.**
*Let $\mathbf{B}$ an axis aligned cube centered at the origin, with side length $L_{\mathbf{B}}$. Let each face be covered in $m$ non-overlapping axis aligned square patches, $\rho_{d,j}$, where the four corners of patchs are $\rho_{d,j,k}$ ($d \in \{\pm x, \pm y, \pm z\}$, $j \in \{0 \ldots (m-1)\}$, $k \in \{0,1,2,3\}$).*
*Let $\mathbf{H} = (H_0, H_1, \ldots, H_{s-1})$ be a list of half plane cuts with $Q = (q_0, q_1, \ldots, q_{s-1})$ be a respective point on the plane cut.*
*If, for every patch $\rho_{d,j}$ there exists a half plane $H_i$, with normal $N_{H_i}$, such that every corner of the patch:*

$$(\rho_{d,j,k} - q_i)\cdot N_{H_i} < 0$$

*then $\cap_i H_i$ creates a compact convex hull that is strictly contained in $\mathbf{B}$.*

Proof. The four points of $\rho_{d,j}$, $(c\rho_{d,j,0}, \rho_{d,j,1}, \rho_{d,j,2}, \rho_{d,j,3})$ create a frustum. A point lying on the patch is defined by $u = \sum_k t_k \rho_{d,j,k}$ with $\sum_k t_k = 1, t_k \geq 0$.
Consider:

$$\begin{array}{rl} & (u-q_i)\cdot N_{H_i} \\ = & ((\sum_k t_k \rho_{d,j,k}) - q_i)\cdot N_{H_i} \\ = & ((\sum_k t_k \rho_{d,j,k}) - (\sum_k t_k) q_i)\cdot N_{H_i} \\ = & \sum_k [t_k(\rho_{d,j,k} - q_i)\cdot N_{H_i}] \\ < & 0 \end{array}$$

Since $t_k \geq 0$ with at least one non-zero $t_k$ and $(\rho_{d,j,k} - q_i)\cdot N_{H_i} < 0$ by assumption. Every point on the face of the cube $\mathbf{B}$ lies on the negative side of some half plane cut, bounding a compact convex hull within.

The SPoIF algorithm grows the fence if the fence patches haven't been secured. In order to consider patches secured, an extra condition is needed to make sure the half plane will keep the patch partitioned as the fence grows.

**Theorem 5.**
*Let $\mathbf{B}$, $\mathbf{H} = (H_0, H_1, \ldots, H_{m-1})$, $\mathbf{Q} = (q_0, q_1, \ldots, q_{s-1})$,*

*$N_{H_i}$, $\rho_{d,j}$, $\rho_{d,j,k}$, be as defined in Theorem 4.*
*Let $\mathbf{B}'$ be an axis aligned cube centered at the origin with side length $L_{\mathbf{B}'} \geq L_{\mathbf{B}}$, $L = \frac{L_{\mathbf{B}'}}{L_{\mathbf{B}}} \geq 1$.*
*If, for some patch $\rho_{d,j}$,*

$$\rho_{d,j,k} \cdot N_{H_i} < 0 \text{ and}$$

$$(\rho_{d,j,k} - q_i) \cdot N_{H_i} < 0$$

*then the patch $L \cdot \rho_{d,j}$ remains outside of the halfspace $H_i$: $L \cdot \rho_{d,j} \cap H_i = \emptyset$.*

Proof. As in Theorem 4, the four points $\rho_{d,j,k}$, ($k \in \{0,1,2,3\}$), with a point lying on the patch now defined by $v = \sum_k L t_k \rho_{d,j,k}$ with $L \geq 1, \sum_k t_k = 1, t_k \geq 0$.

$$\begin{aligned} & (v - q_i) \cdot N_{H_i} \\ = \; & ((\textstyle\sum_k L t_k \rho_{d,j,k}) - q_i) \cdot N_{H_i} \\ = \; & ((\textstyle\sum_k L t_k \rho_{d,j,k}) - (\textstyle\sum_k t_k) q_i) \cdot N_{H_i} \\ = \; & \textstyle\sum_k [(L-1) t_k \rho_{d,j,k} \cdot N_{H_i} + t_k (\rho_{d,j,k} - q_i) \cdot N_{H_i}] \\ < \; & 0 \end{aligned}$$

Both terms in the sum are negative by assumption of $\rho_{d,j,k} \cdot N_{H_i} < 0$ and $(\rho_{d,j,k} - q_i) \cdot N_{H_i} < 0$, where the coefficients $(L-1)$ and $t_k$ are non-negative with at least one $t_k$ positive.

Theorem 4 ensures that a compact convex hull in contained within the fence when the fence patches have been partitioned. Theorem 5 ensures that once a patch has been secured, it remains secured should the fence need to grow. Theorem 5 allows us to secure patches with candidate nearby points to an anchor point and not reprocess the candidate points as they secure fence patches. There may be optimizations that reuse candidate nearby points but these are not pursued further in this paper.

**Theorem 6.** *The function RNGPointSPoIF(P, p) runs in expected $O(1)$ time*

Proof. ...

Finding the relative neighborhood graph connections for $p$ is $O(1)$, providing us with the desired expected linear time algorithm:

**Theorem 7.** *The function RNGSPoIF(P) runs in expected $O(N)$ time*

TODO:
- securing fence implies compact convex hull on interior
- a secured fence need only consider a larger fence, of factor $\sqrt{3}$, to include all points that could influence relative neighborhood edges to $p$
- $SPoIF : RNGp(P,p)$ runs in constant expected time
- $SPoIF : RNG(P)$ runs in expected linear time

## 5. Conclusion

An expected linear time algorithm to find the relative neighborhood graph for 3D points placed uniformly at random in the unit cube. To the author's knowledge, no well known expected linear time algorithm for finding the relative neighborhood graph is known for the case of points distributed uniformly in the unit cube.

Randomness of point distribution eliminates many pathological cases that would need to be taken care of with a fully deterministic algorithm. The bounded maximum degree of the relative neighborhood graph in 3D, along with the randomness assumption, allow a construction of an expected linear time algorithm by employing a coarse plane partition heuristic that allows only local neighborhoods of points to be considered. A finite neighborhood makes naive calculations constant time, allowing for an overall expected linear runtime algorithm.

The algorithm presented also answers a question posed by Katajainen and Nevalainen [**?** ] of whether there's a simpler algorithm that wouldn't need to handle points close to the border differently. The algorithm presented avoids needing to consider side or corner regions independently as the the plane partition heuristic provides an orthogonal partition and fences near the edge of the region will be secured by falling out of bounds.

Certain pathological simple geometries can cause problems for the algorithm as presented. For example, a "diamond" shaped distribution might have trouble securing the fence. Extensions have the potential to overcome limitations while still retaining overall linear time execution by, for example, doing a pre-processing step that marks grid cells as out of bounds if they fall outside of some initial convex hull calculation, or doing random rotations and processing parallel copies. These problems and ideas are not pursued in this paper other than to mention they exist and there might be a way to mitigate them.

## 6. Bibliography

## A. Appendix.0

For completeness, this section discusses the elementary, but tedious, calculations for the dart shape and volume. As a reminder, our goal is to show that increasing the grid fence by a fixed size will almost surely encapsulate a compact convex hull made from an anchor point and its neighbors. The convex hull guarantees that the relative neighborhood graph edges of the anchor point to its neighbors need only consider candidate points within the convex hull (with some caveats about extending the grid fence to include saboteurs, detailed in section **??**).

The logic chain is roughly:

- For an anchor point, $p$, and a grid fence side, there will be a region that a neighbor point can fall in that will partition the fence side from the point, a process we call *securing the fence*, where a secured fence necessarily implies a compact convex hull made from the points on the interior
- The volume that secures a fence side will turn out to be four sphere slices joined together, which we will call a *dart*, where a sphere slice is the intersection of two perpendicular half planes and a sphere
- A lower bound estimate on the dart volume is a constant proportion of overall grid fence volume
- A non-trivial constant dart volume makes the question of how many neighbors to be expected to secure the overal six sided fence into a coupon collector-like problem, giving us an expected finite grid fence size
- Finally, approximatations on the four slices bundled together allow us to give coarse values for grid fence sizes

It should be stressed that the discussion and estimates are meant primarily to show existence of a finite grid fence size. We focus on securing a fence side by waiting for some single neighbor point to secure it. The Shrinking Posts on Increasing Fence (SPoIF) algorithm in this paper has a smaller expected grid fence size before securitization as points can secure parts of the fence through clusters of posts, increasing the admissible region on each face side that can be secured.

The calculations done in this appendix are meant to show that we only need to consider a finite grid fence size by providing correct, if horrible, bounds. In this paper, we are satisfied with providing empherical results for grid sizes that SPoIF uses without going into more rigorous analysis.

First consider a fixed point $q$ and a fixed but moveable point $\eta$.

We think of $\eta$ as a point on a plane with normal $\eta/|\eta|$ that intersects $q$ in the slice of $z = 0$ to $z = 1$.

As we vary $\eta$ we want to know what the shape is traced out.

The claim is that this is the surface of a sphere whose center is at $q/2$ of radius $q/2$.

To see this, consider $\eta = q/2+(|q|/2)v$, with $|v| = 1$.

Calculate dot product of the line from $q$ to $\eta$ with the line from the origin to $\eta$:

$$\begin{aligned} & \eta = q/2 + (|q|/2)v \\ & (q/2 + (|q|/2)v) \cdot (q/2 + (|q|/2)v - q) \\ = \; & (q/2 + (|q|/2)v) \cdot (-q/2 + (|q|/2)v) \\ = \; & -(1/4)|q|^2 + (|q|/2)^2|v|^2 \\ = \; & 0 \end{aligned}$$

Proving the claim.

We want to find the the minimum volume of the dart for some point inside of a fence of radius $s$ (side length $2s$) nested inside of a larger fence of radius $s(2k+1)$ (side length $2s(2k+1)$).

First consider a sphere of radius $R$, centered at $(-D, -D, -D)$ intersected with two half planes, $H^+_{x=0} = \{u : u_x \geq 0\}$, $H^+_{y=0} = \{u : u_y \geq 0\}$. For suitable $R$ and $D$, we want to estimate the volume.

If we take two tetrahedrons completely enclosed by the slice, the maximum diagonal of the slice is $(R-\sqrt{2}D)$, with maximum straight side length (in the $x$-axis line and $y$-axis line) to also be $(R-\sqrt{2}D)$ and slice half height to also be $(R-\sqrt{2}D)$.

The volume of the tetrahedron is then:

$$(1/6)(R-\sqrt{2}D)^2\sqrt{R^2-2D^2}$$

The volume of the two tetrahedrons stacked on top of each other:

$$(1/3)(R-\sqrt{2}D)^2\sqrt{R^2-2D^2}$$

More succinctly:

$$V_{\text{slice}} > \tfrac{1}{3}(R-\sqrt{2}D)^2\sqrt{R^2-2D^2}$$

We will get concrete estimates below but as a quick confirmation, we can check to make sure that this slice is a constant proportion of the fence it sits in.

We know that $R \propto (ks)$ and $D \propto (ks)$, for some $k \in \mathbb{Z}$ and grid cell size $s \in \mathbb{R}$. Call $R = \alpha(ks)$ and $D = \beta(ks)$.

The dart will be four slices bundled together sitting inside of a $\gamma^3(ks)^3$ volume. Take $\alpha, \beta \in \mathbb{R}$ to be the smallest of the four slices:

$$\begin{aligned} & \frac{(4/3)(\alpha(ks)-\sqrt{2}\beta(ks))^2\sqrt{\alpha^2(ks)^2-2\beta^2(ks)^2}}{\gamma^3(ks)^3} \\ = \; & \frac{(4/3)(ks)^3(\alpha-\sqrt{2}\beta)^2\sqrt{\alpha^2-2\beta^2}}{\gamma^3(ks)^3} \\ = \; & \frac{(4/3)(\alpha-\sqrt{2}\beta)^2\sqrt{\alpha^2-2\beta^2}}{\gamma^3} \end{aligned}$$

as desired.

When choosing random points inside of the larger fence cube, we can think of the dart as the probability a single point secures one side of the fence. It is equivalent to talk about securing one side of the fence face with securing four posts on each of corners of the fence face. As a reminder, the Shrinking Posts on Increasing Fence (SPoIF) uses nine posts per face, equally distributed along the face square. Since there are more opportunities to secure clusters of posts on a fence face, in addition to still retaining the features of the dart region, SPoIF will be able to secure the fence cube more quickly. We only consider the four post fence here to help simplify the calculation even though

it inflates the expected number of points we need to see to secure the grid fence.

Take the minimum dart value volume of the six directions and call it $V_{\text{dart}}$. The probability that a point lands in one of the darts is bounded below by $V_{\text{dart}}/V_{\text{fence}}$. Of the points that land in one of the darts, the time we need to take to secure each face of the fence is now a coupon collector-like problem with six coupons.

We can now plug in some real numbers to get coarse bounds on dart volumes, probabilities of landing in the critical region and expected grid sizes.

As a reminder, we are considering a center grid cell, of side length $s$ where the anchor point, $p$, resides, all of which is inside a larger grid fence of side length $2s(k+1)$, for some fixed $k \geq 0, k \in \mathbb{Z}$ as yet to be determined.

The smallest size a slice can be is if the anchor point is on the corner of the interior grid cell and to choose the grid fence corner nearest to it. For example, $p = (s/2, s/2, s/2)$, $q = (s(k+1/2), s(k+1/2), s(k+1/2))$.

In such a case, the center point of the sphere is:

$$(p+q)/2 = (s(k+1)/2, s(k+1)/2, s(k+1)/2)$$

with radius:

$$\begin{aligned} R &= |(p+q)/2 - p| \\ &= |(sk/2, sk/2, sk/2)| \\ &= (\sqrt{3}/2)sk \end{aligned}$$

Here:

$$\begin{aligned} V_{\text{fence}} &= (s(2k+1))^3 \\ D &= s(k+1)/2 - s/2 \\ &= sk/2 \end{aligned}$$

This yields:

$$\begin{aligned} \frac{V_{\text{dart}}}{V_{\text{fence}}} &> \frac{\frac{4}{3}(\frac{\sqrt{3}}{2}sk - \frac{\sqrt{2}}{2}sk)^2(\frac{3}{4}s^2k^2 - \frac{1}{2}s^2k^2)^{\frac{1}{2}}}{s^3(2k+1)^3} \\ &= \frac{s^3k^3(\sqrt{3}-\sqrt{2})^2}{3\sqrt{2}s^3(2k+1)^3} \\ &= \frac{(\sqrt{3}-\sqrt{2})^2}{3\sqrt{2}(2+\frac{1}{k})^3} \\ &> \frac{(\sqrt{3}-\sqrt{2})^2}{81\sqrt{2}} \qquad (k \geq 1) \\ &> 1/1198 \end{aligned}$$

For points that land within one of the six dart region, we would expect this to be be a six coupon collector problem with, at worst, equal probability. The coupon collector problem grows roughly as $n\log(n)$ which, in our case of six sides of the fence to secure, gives us an expected 11 or more points before we secure the fence.

Each point has roughly a $6/1198$ chance of landing in one of the six darts, so we would expect roughly $11/(6/1198) = 2196.3\ldots$ number of points within the grid fence to appear before securing the fence.

Each grid cell has roughly one point, so we would expect the number grid cells on a side to be

$$\begin{aligned} &(11 \cdot 1198/6)^{\frac{1}{3}} \\ &= 12.9\ldots \\ &\sim 13 \end{aligned}$$

That is, we would expect $k \sim 6$, iterating roughly six to seven steps of increasing the grid fence size until securing the fence in this way.

## B. Appendix.1

... Appendix sections are coded under `\appendix`.

## References

...